Fall 2026 Midterm

ASEN 5090 Introduction to GNSS

Justin Le
October 23, 2026

Exam Info

Start Date & Time

October 23rd, 2026, 1130

End Date & Time

Honor Code

On my honor, as a University of Colorado Boulder student, I have followed all of the above instructions and have neither given nor received unauthorized assistance on this work.
Justin Le, October 23rd, 2026

Problem 1:

(a.)

Because both orbits are circular, we can simply use the formula to solve for the orbital velocity of a satellite in a circular orbit with the gravitational constant for Earth, $\mu_E = 398600.5 \text{ km}^3/\text{s}^2$,

$$v = \sqrt{\frac{\mu_E}{r}},\ r_{GPS} = 26560 \text{ km},\ r_{LEO} = 7180 \text{ km}$$

Solving for each radius of the respective satellites,

$$\boxed{v_{GPS} = 3.87 \text{ km/s}}$$

$$\boxed{v_{LEO} = 7.45 \text{ km/s}}$$

(b.)

The minimum geometrical range for a satellite with some observer on the equator of the Earth would be when the satellite directly passes overhead the observer at an elevation of $el = 90°$ we can then account for this with the geometrical range equation,

$$R = \sqrt{R_E^2(\sin^2 el - 1) + R_{sat}^2} - R_E \sin el,\ el = 90°$$

We use the corresponding ranges from LEO and GPS to solve,

$$\boxed{\rho_{GPSmin} = 20180 \text{ km}}$$

$$\boxed{\rho_{GPSmin} = 800 \text{ km}}$$

(c.)

To find the maximum geometrical range for which the satellite is visible, we want to account for when the elevation is 0° for each satellite. Using the same equation in part b but with an elevation of $el = 0°$,

$$\boxed{\rho_{GPSmax} = 2.578 \times 10^4 \text{ km}}$$

$$\boxed{\rho_{GPSmax} = 3.294 \times 10^3 \text{ km}}$$

(d.)

When assuming a non rotating Earth, perfect satellite and user clocks, and clear view all the way down,the doppler shift of a satellite transmitting at the minimum geometrical range would be 0 Hz for both satellites. Because doppler frequency is dependent on the range rate of the satellite, at this point the range would be a minimum which leads to zero doppler shift.

(e.)

With similar assumptions listed in part d, when calculating the doppler shift at the maximum geometrical range, the doppler shift will be inherently bounded by the orbital speed of the satellite, which will be different for each GPS and LEO satellite scenario. Finding the doppler shift at some elevation of 0 we use,

$$f_D = \frac{v_{sat}}{c}\frac{R_{obs}}{R_{sat}}f_{carrier}$$

Substituing the corresponding velocities found in part a, the geometric ranges in part b, and the given carrier fre

$$\boxed{f_{DGPS} = 4.87 \text{ kHz} = 4.87 \times 10^3 \text{ Hz}}$$

$$\boxed{f_{DLEO} = 176.55 \text{ Hz} = 176.55 \times 10^3 \text{ Hz}}$$

	GPS	LEO
Satellite speed (magnitude of velocity)	3.87 km/s	7.45 km/s
Minimum geometrical range	20180 km	800 km
Maximum geometrical range for which the satellite is visible	2.578×10^4 km	3.294×10^3 km
Doppler shift at minimum geometrical range	0 Hz	0 Hz
Doppler shift at maximum geometrical range	4.87×10^3 Hz	176.55×10^3 Hz

Table 1: GPS vs LEO

(f.)

When comparing the pseudorange ionospheric delays between the two satellites, we can use the equation that approximates the first order terms which models the ionospheric effects due to TEC,

$$I_\rho = \frac{40.3TEC}{f_{carrier}^2}$$

When both satellites are directly overhead at zenith, the TEC at LEO satellite is at zenith would be smaller than the TEC at the GPS satellite. Taking that into account and also the fact that the frequency for the LEO sat is greater than the GPS satellites' carrier frequency, the ionospheric effect of the LEO sat should be much less than the GPS satellite's.

(g.)

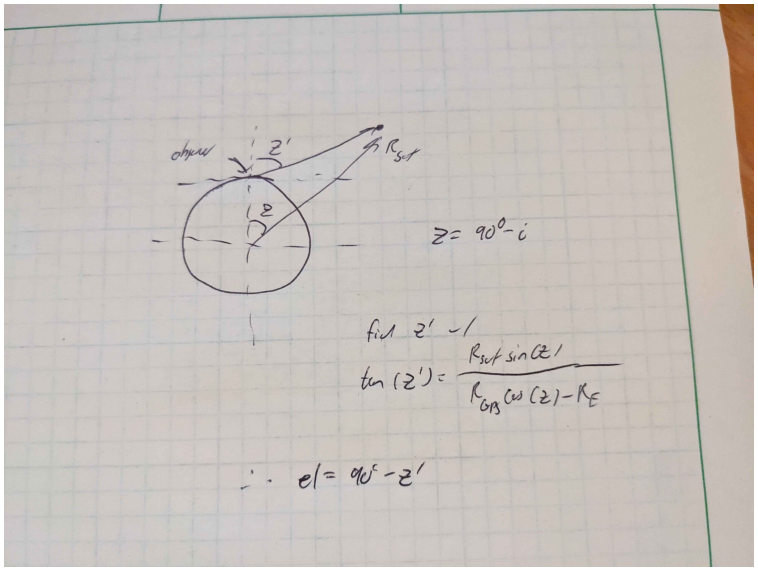


Figure 1: North Pole Visibility Diagram

Visibility can be calculated based on what can be seen in Figure 1, and the equation,

$$\tan z' = \frac{R_{sat} \sin z}{R_{sat} \cos z - R_E}$$
$$z = 90° - i$$
$$el = 90° - z'$$

We can use arctan to find z' for LEO and GPS, we can then find their respective elevations.

$$\boxed{el_{LEONorthPole} = -6.9°}$$
$$\boxed{el_{GPSNorthPole} = 45.3°}$$

This means that through the entire orbit of the LEO sat, an observer on the North Pole would never be able to observe it as the maximum elevation that it would ever reach relative to the North Pole would be below the horizon. By contrast, the GPS satellite will be observed though at a maximum of 45.3° in elevation, essentially only half of the sky.

MATLAB Code

```
%% ASEN 5090 GPS/GNSS
%% Justin Le
%% FALL 2026 MIDTERM SCRIPT PART 1

clc;clear;close all

% constants
c = 3E8;% m/s
mu_E = 398600.5; %km^3/s^2

%% Problem 1:
% a. Find satellite speed

% constatns
R_E = 6380; %km

r_GPS = 26560; %km
i_GPS = 55; %deg

r_LEO = R_E + 800; %km
i_LEO = 55; %deg

f_GPS = 1575.42E6; %Hz, GPS L1 band
f_LEO = 8E9; %Hz

% Use formula for circular orbits
v_GPS = sqrt(mu_E/r_GPS)
v_LEO = sqrt(mu_E/r_LEO)

% b. Calculate minimum geometrical range

% Assume an observer on the equator where satellites pass overhead.
rObsLLA = [0,0,0];
rObsECEF = lla2ecef(rObsLLA, 0, 6380E3) % m
rObsECEF = rObsECEF./1000 % km

rObs = norm(rObsECEF);
el=90;
r_min_diff_GPS = sqrt( rObs.^2*(sind(el).^2-1) + r_GPS.^2) - rObs*sind(el)
r_min_diff_LEO = sqrt( rObs.^2*(sind(el).^2-1) + r_LEO.^2) - rObs*sind(el)


% c. Calculate maximum geometrical range, with elevation = 0
el=0;
r_min_diff_GPS = sqrt( rObs.^2*(sind(el).^2-1) + r_GPS.^2) - rObs*sind(el)
r_min_diff_LEO = sqrt( rObs.^2*(sind(el).^2-1) + r_LEO.^2) - rObs*sind(el)

% e. Calculate doppler shift at elevation of 0
f_D_GPS = (v_GPS/c)*(R_E/r_GPS) * f_GPS
f_D_LEO = (v_LEO/c)*(R_E/r_LEO) * f_LEO

% g. Find if north pole can ever observe the LEO sat
```

```
z_LEO = 90-i_LEO;
z_GPS = 90-i_GPS;

z_max_LEO = atan2d(r_LEO*sind(z_LEO), r_LEO*cosd(z_LEO) - R_E)

z_max_GPS = atan2d(r_GPS*sind(z_GPS), r_GPS*cosd(z_GPS) - R_E)

el_LEO = 90-z_max_LEO

el_GPS = 90-z_max_GPS
```

Problem 2:

(a.)

TX	G-matrix				Expected range	Prefit residual
A	0	-0.9864	0.1644	1	912.4 m	48.6 m
B	-0.6247	0.6247	0.4685	1	320.2 m	-133.4 m
C	0.7593	0	0.6508	1	461 m	100.4 m
D	-0.4286	-0.2857	0.8571	1	700 m	-51.2 m
E	0	0	0	1	600 m	-9.5 m

Table 2: G-matrix, expected ranges, and prefit residuals

(b.)

To solve for an updated estimate, we can simply calculate for the correction vector,

$$\begin{bmatrix} \delta\hat{x} \\ \delta\hat{b} \end{bmatrix} = (G^T G)^{-1} G^T \delta\rho$$

As a column vector on the first iteration we would find it to be,

$$\begin{bmatrix} \delta\hat{x} \\ \delta\hat{b} \end{bmatrix} = \begin{bmatrix} 142.2 \\ -58.1 \\ 0.224 \\ -8.34 \end{bmatrix}$$

Apply this to the first guess at the origin to find an updated position and clock bias of,

$$R_{new} = \begin{bmatrix} 142.1564 \text{ m} \\ -58.1 \text{ m} \\ 0.224 \text{ m} \end{bmatrix}$$
$$b = -8.34 \text{ m}$$

(c.)

Iteration Count	X (m)	Y (m)	Z (m)
1	142.15	-58.1	0.222
2	143.7997	-60.3104	11.3977
3	143.7956	-60.2514	11.2693
4	143.7956	-60.2514	11.2692

Table 3: Iteration Results

MATLAB Code

```
%% ASEN 5090 GPS/GNSS
%% Justin Le
%% FALL 2026 MIDTERM SCRIPT PART 2

clc;clear;close all

% constants
c = 3E8;% m/s
mu_E = 398600.5; %km^3/s^2

%% Problem 2: Pseudoranging
% Givens
tx.A.positions = [0,-900,150];
tx.B.positions = [-200,200,150];
tx.C.positions = [350,0,300];
tx.D.positions = [-300,-200,600];
tx.E.positions = [0,0,600];

tx.A.rho = 863.8;
tx.B.rho = 453.6;
tx.C.rho = 360.6;
tx.D.rho = 751.2;
tx.E.rho = 609.5;

R_obs = [0,0,0];

%compute_least_point_solution

% Find Geometry matrix

tx_names = fieldnames(tx);

for i = 1:numel(tx_names)
    tx_name = tx_names{i};
    R0 = sqrt((tx.(tx_name).positions(1) - R_obs(1)).^2 + (tx.(tx_name).positions(2) -
        R_obs(2)).^2 + (tx.(tx_name).positions(3) - R_obs(3)).^2);
```

```
    G(i,:) = [(tx.(tx_name).positions(1) - R_obs(1))./R0, (tx.(tx_name).positions(2) -
        R_obs(2))./R0, (tx.(tx_name).positions(3) - R_obs(3))./R0]
end

G = [G, ones(length(fieldnames(tx)),1)]

[G,dx_hat] = least_squares(tx, R_obs)

% Calculate expected range
for i = 1:numel(tx_names)
    tx_name = tx_names{i};
    tx.(tx_name).expected_range = norm(tx.(tx_name).positions - R_obs);
end
disp("Expected Ranges")
disp(tx.A.expected_range)
disp(tx.B.expected_range)
disp(tx.C.expected_range)
disp(tx.D.expected_range)
disp(tx.E.expected_range)

% Calculate prefit residuals
for i = 1:numel(tx_names)
    tx_name = tx_names{i};
    tx.(tx_name).delta_rho_pre = tx.(tx_name).expected_range - tx.(tx_name).rho;
end

disp("Prefit Residuals")
disp(tx.A.delta_rho_pre)
disp(tx.B.delta_rho_pre)
disp(tx.C.delta_rho_pre)
disp(tx.D.delta_rho_pre)
disp(tx.E.delta_rho_pre)

% b. Solve for updated estimate of receiver position and clock bias
% Get prefit residuals into a column vector
for i = 1:numel(tx_names)
    tx_name = tx_names{i};
    delta_rho_pre(i,1) = tx.(tx_name).delta_rho_pre;
end

% Finding corrections
dx_hat = (G'*G)\(G'*delta_rho_pre);

% Applying corrections
R_obs_prime = R_obs' + dx_hat(1:3)

%% Iterate solution 3 more times. (4 times total)
clc;clear

% Givens
tx.A.positions = [0,-900,150];
tx.B.positions = [-200,200,150];
tx.C.positions = [350,0,300];
tx.D.positions = [-300,-200,600];
```

```
tx.E.positions = [0,0,600];

tx.A.rho = 863.8;
tx.B.rho = 453.6;
tx.C.rho = 360.6;
tx.D.rho = 751.2;
tx.E.rho = 609.5;

R_est = [0,0,0];

for i = 1:4

    [G,dx_hat] = least_squares(tx,R_est)

    R_est = R_est + dx_hat(1:3)';


    disp("R_est:")
    disp(R_est)
end

function [G,dx_hat] = least_squares(tx, R_obs)
tx_names = fieldnames(tx);
    for i = 1:numel(tx_names)
        tx_name = tx_names{i};
        R0 = sqrt((tx.(tx_name).positions(1) - R_obs(1)).^2 + (tx.(tx_name).positions(2) -
             R_obs(2)).^2 + (tx.(tx_name).positions(3) - R_obs(3)).^2);
        G(i,:) = [(tx.(tx_name).positions(1) - R_obs(1))./R0, (tx.(tx_name).positions(2) -
             R_obs(2))./R0, (tx.(tx_name).positions(3) - R_obs(3))./R0];
    end
G = [G, ones(length(fieldnames(tx)),1)];

% Calculate expected range
for i = 1:numel(tx_names)
    tx_name = tx_names{i};
    tx.(tx_name).expected_range = norm(tx.(tx_name).positions - R_obs);
end

disp("Expected Ranges")
disp(tx.A.expected_range)
disp(tx.B.expected_range)
disp(tx.C.expected_range)
disp(tx.D.expected_range)
disp(tx.E.expected_range)

% Calculate prefit residuals
for i = 1:numel(tx_names)
    tx_name = tx_names{i};
    tx.(tx_name).delta_rho_pre = tx.(tx_name).expected_range - tx.(tx_name).rho;

    delta_rho_pre(i,1) = tx.(tx_name).delta_rho_pre;
end
disp("Prefit Residuals")
disp(tx.A.delta_rho_pre)
```

```
disp(tx.B.delta_rho_pre)
disp(tx.C.delta_rho_pre)
disp(tx.D.delta_rho_pre)
disp(tx.E.delta_rho_pre)

% Finding corrections
dx_hat = (G'*G)\(G'*delta_rho_pre);

end
```

Problem 3:

(a.)

Code A would indeed be useful for ranging. This is because the big factor for these codes would be being able to differentiate between what ranging information is being sent. This is done by having some code have high autocorrelation. Here we see that the highest spike in correlation occurs when the code is not shifted and when the code is shifted 15 bits (effectively looping around). This is indicative of high autocorrelation.

(b.)

To find the correlation between a Code A replica and a transmitted Coda A from 600 meters away, we find the time it takes to reach the receiver and account for the chip shift according to the chipping rate,

$$t = 600 \text{ m} * 1/c, \; c = 3 \times 10^8 \text{ m/s}$$
$$t = 2 \times 10^{-6} \text{ s} = 2 \; \mu\text{s}$$

$$chip_{rate} = 6 \times 10^6 \text{ Hz} = 6 \times 10^6 \text{ chips/s}$$
$$shift = 6 \times 10^6 \text{ chips/s} * 2 \times 10^{-6} \text{ s}$$
$$\therefore shift = 12 \text{ chips}$$

Therefore, we can shift Code A by 12 chips and use the correlation function to find,

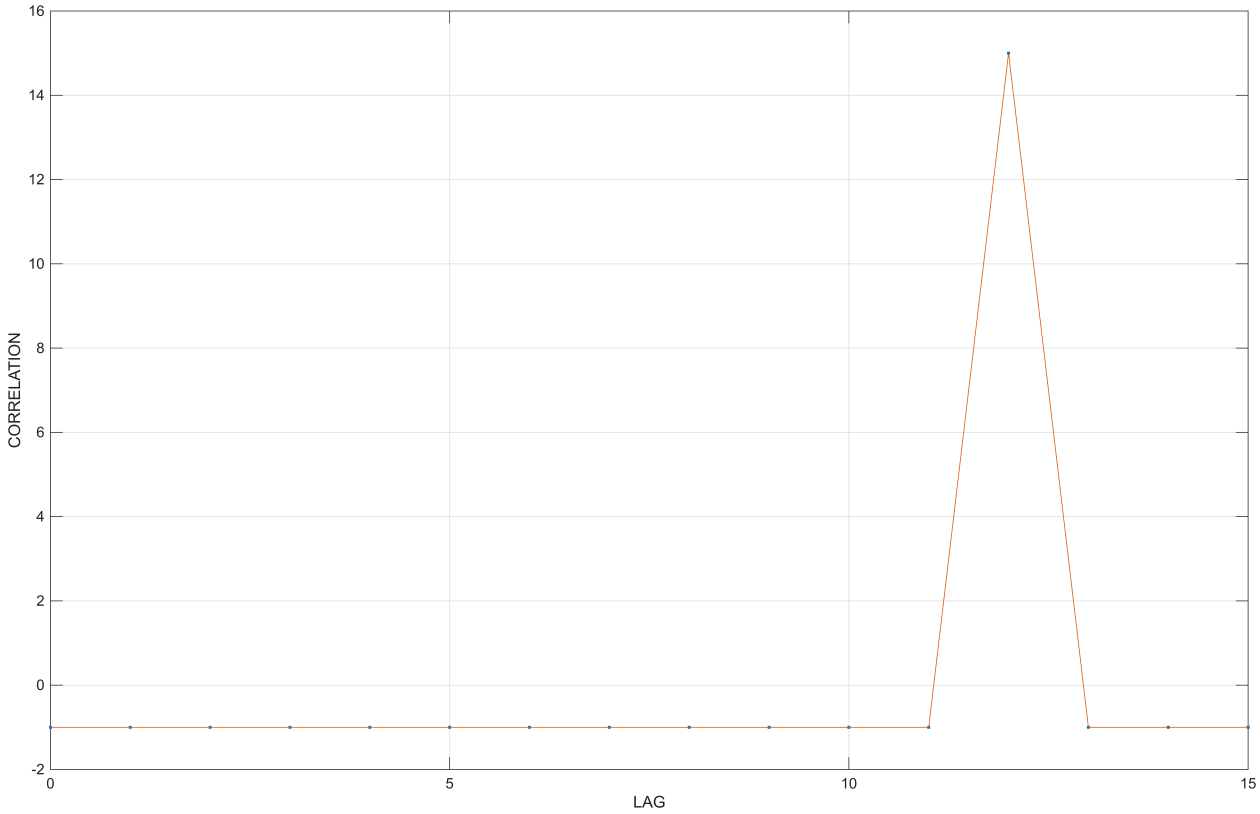


Figure 2: Correlation of Code A Shifted by 12 chips

(c.)

Using the correlation function once again to find the cross correlation between Code A and Code B we find,

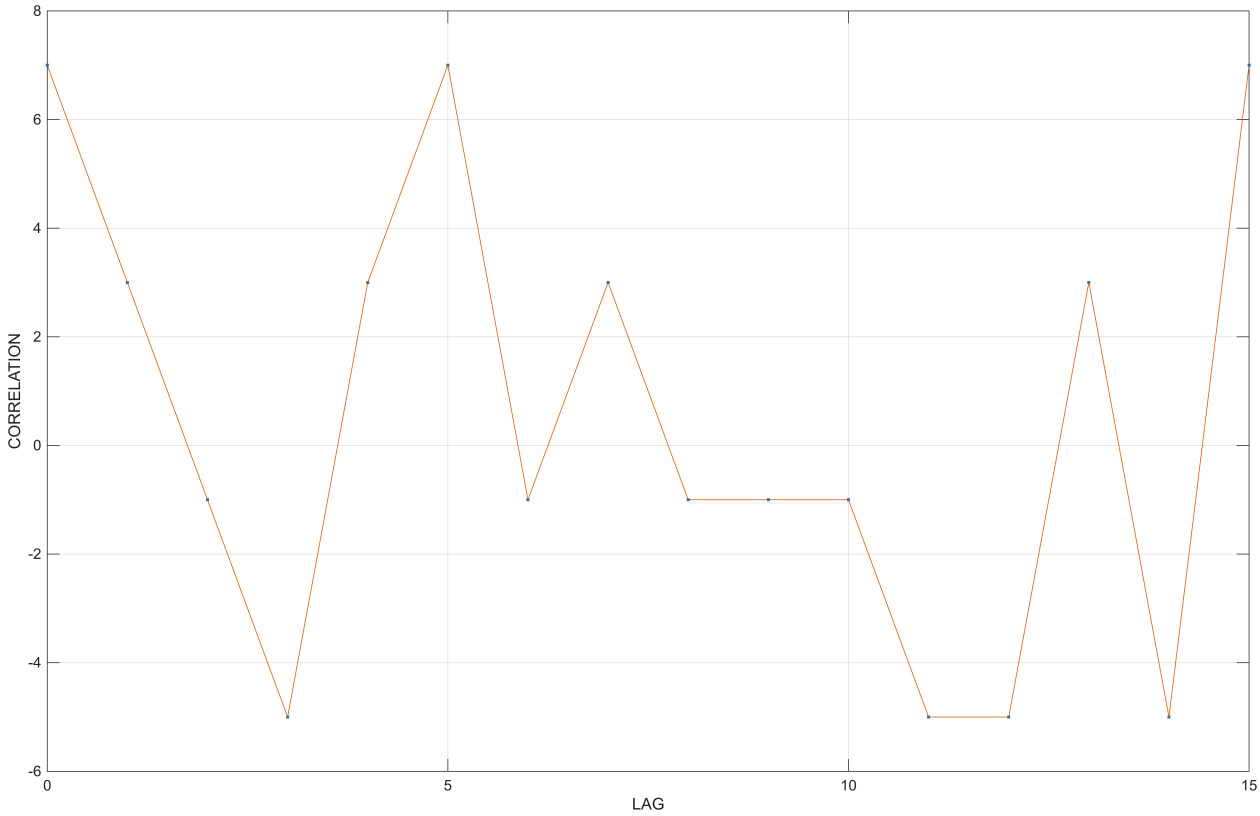


Figure 3: Cross Correlation Between A and B

(d.)

This pair of codes for CDMA would be not that useful for CDMA. CDMA would want high auto correlation and low cross correlation. The pair of codes here would indeed have good auto correlation as observed with code A prior and Figure 4 is the autocorrelation for code B. However when examning the cross correlation between them, there is no significant spike throughout but looking at the scale we see that there max heights are about half of their respective autocorrelation spikes which is relatively large. When trying to use this when both codes are being transmitted there is chance of false correlations between them making this pair of codes a poor choice for CDMA.

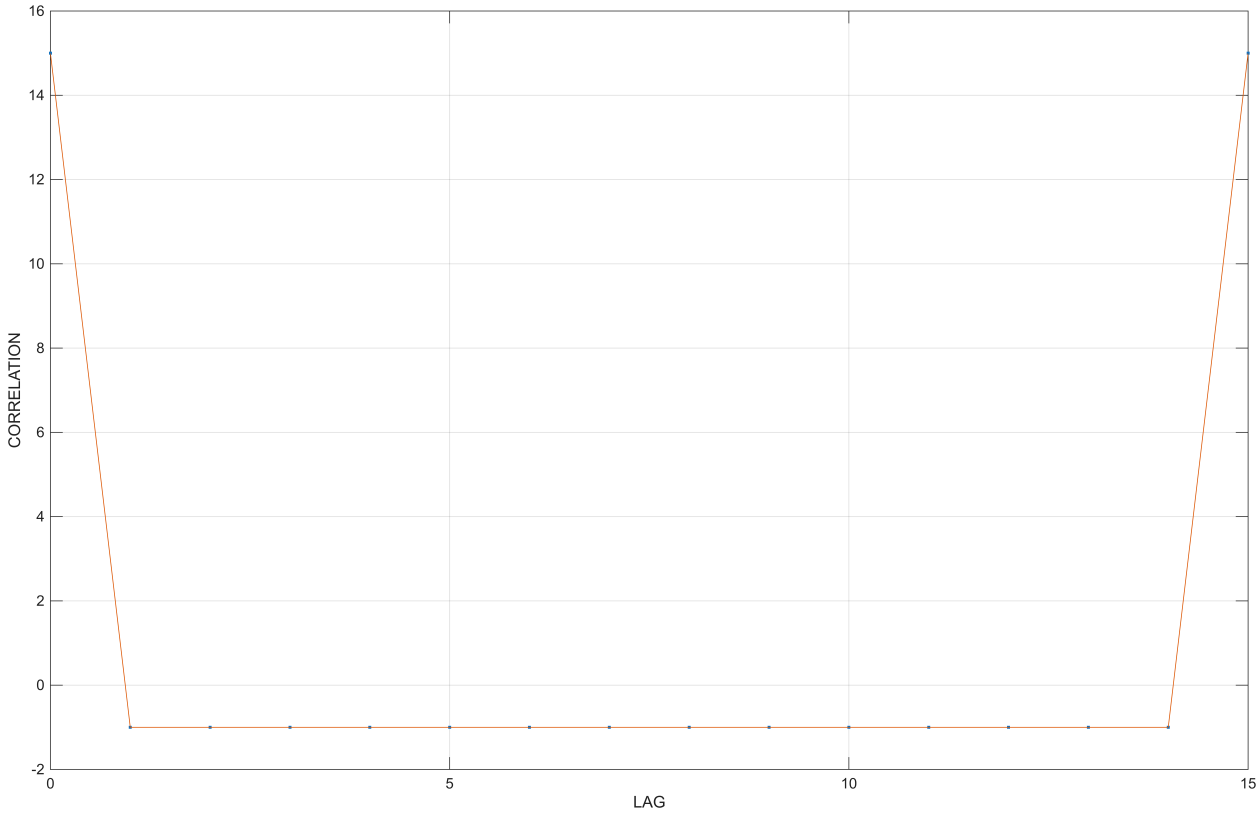


Figure 4: Code B Autocorrelation

MATLAB Code

```
%% ASEN 5090 GPS/GNSS
%% Justin Le
%% FALL 2026 MIDTERM SCRIPT PART 3

clc;clear;close all
% constants
c = 3E8;% m/s

rate = 6E6; % Hz

codeA = [-1,-1,-1,-1, 1, 1, 1,-1, 1, 1,-1,-1, 1,-1, 1];
codeB = [-1,-1,-1,-1, 1,-1, 1,-1,-1, 1, 1,-1, 1, 1, 1];

figure()
cyc_corr_basic(codeA, codeA)
exportgraphics(gcf, fullfile('figures', 'Midterm', 'codeA_autocorrelate.png'), '
    Resolution', 900);
cyc_corr_basic(codeB, codeB)
exportgraphics(gcf, fullfile('figures', 'Midterm', 'codeB_autocorrelate.png'), '
    Resolution', 900);

%b. Correlation of Code A 600 m away

t = 600 * (1/c)
chip_shift = rate*t


% Shift by 12 chips
codeA_12 = codeA([end-11:end 1:end-12]);

figure()
cyc_corr_basic(codeA_12, codeA)
exportgraphics(gcf, fullfile('figures', 'Midterm', 'codeA_autocorrelate_shift12.png'), '
    Resolution', 900);


% c. Finding cross correlation
cyc_corr_basic(codeB, codeA)
exportgraphics(gcf, fullfile('figures', 'Midterm', 'cross_correlationAB.png'), '
    Resolution', 900);
```